MINISTÉRIO DA DEFESA • EXÉRCITO BRASILEIRO • IME

Curso de Graduação em Engenharia da Computação

Projeto de Final de Curso · 2026

Planejamento de Trajetória para Robô Terrestre usando Grafos

Progresso do projeto: da Verificação de Especificação (VE) à versão atual

Autores

Gabriel Dias Vilela • **Tassandro Cavalcante Leitão**

Orientadores

Profa. Claudia Marcela Justel, D.Sc. • Prof. Paulo Fernando Ferreira Rosa, Ph.D.

Rio de Janeiro • 2026

Onde paramos: a Verificação de Especificação

O que já estava consolidado na VE

- **Arquitetura de software**: modular e orientada a objetos, em cinco módulos encadeados (ingestão → pré-processamento → grafo → algoritmo → pós-processamento)
- **Algoritmos de busca em grafos**: BFS, DFS, Dijkstra e A* implementados sobre uma representação genérica de grafo
- **Cálculo da rota na imagem**: sobre um mapa de navegabilidade com cores fixas, convertido em grafo pixel a pixel
- **Ainda como propostas**: a segmentação semântica e a validação em simulação eram descritas, mas não estavam implementadas

O que evoluiu desde a VE

Três frentes de progresso, sobre a mesma arquitetura modular

	VE (verificação)	Versão atual
Algoritmos	BFS, DFS, Dijkstra, A*	+ RRT (planejamento por amostragem)
Áreas transitáveis	Mapa externo / cores fixas	Segmentação semântica U-Net no pipeline
Validação	Ausente	ROS 2 + Gazebo: integração e simulação (trajetórias de teste)

Cada frente foi adicionada sob as interfaces já existentes, sem reescrever o restante do sistema.

O que é o RRT?

Árvore de Exploração Rápida Aleatória (Rapidly-exploring Random Tree)

- **Ideia central**: constrói incrementalmente uma árvore a partir do ponto inicial, explorando o espaço livre por amostragem aleatória
- **A cada iteração**: (1) sorteia um ponto aleatório; (2) encontra o nó da árvore mais próximo; (3) avança um passo fixo (step size) na direção do ponto; (4) adiciona o novo nó se o segmento estiver livre de colisão
- **Término**: quando um novo nó tem linha de visada desobstruída até o objetivo
- **Propriedades**: probabilisticamente completo (encontra uma solução, se existir, dado tempo suficiente), mas não ótimo: o caminho é irregular e depende da aleatoriedade
- **Compromissos**: baixo custo por iteração, o que o torna rápido; em contrapartida, tem dificuldade em passagens estreitas

Aprimoramento 1: Inclusão do RRT

Rapidly-exploring Random Tree: planejamento por amostragem

- **Novo planejador no catálogo**: adicionado ao lado de BFS, DFS, Dijkstra e A*, sob a mesma interface
- **Espaço contínuo sobre o mapa**: cresce uma árvore por amostragem aleatória
- **Probabilisticamente completo, porém não ótimo**: usado como comparação frente à busca em grafos, retomando a análise iniciada no PFC 2025
- **Parâmetros expostos**: step size, número de iterações e semente aleatória (reprodutibilidade), na linha de comando e na interface gráfica

RRT frente à busca em grafos

Mesmo mapa, mesmos pontos de origem e destino



- **A* (azul escuro) e Dijkstra (verde)**: mesma solução ótima; o A* explora menos nós (203 mil vs. 228 mil)
- **BFS (rosa)**: menor número de arestas, mas passa mais perto de obstáculos (maior custo ponderado)
- **RRT (azul claro)**: de longe o mais rápido (~0,01 s), porém não ótimo e sujeito à variabilidade estocástica
- **DFS (amarelo)**: Claramente sub-otimo, pois trafegou por grande faixa de zona transitável até atingir o destino (gerando o pior caminho).

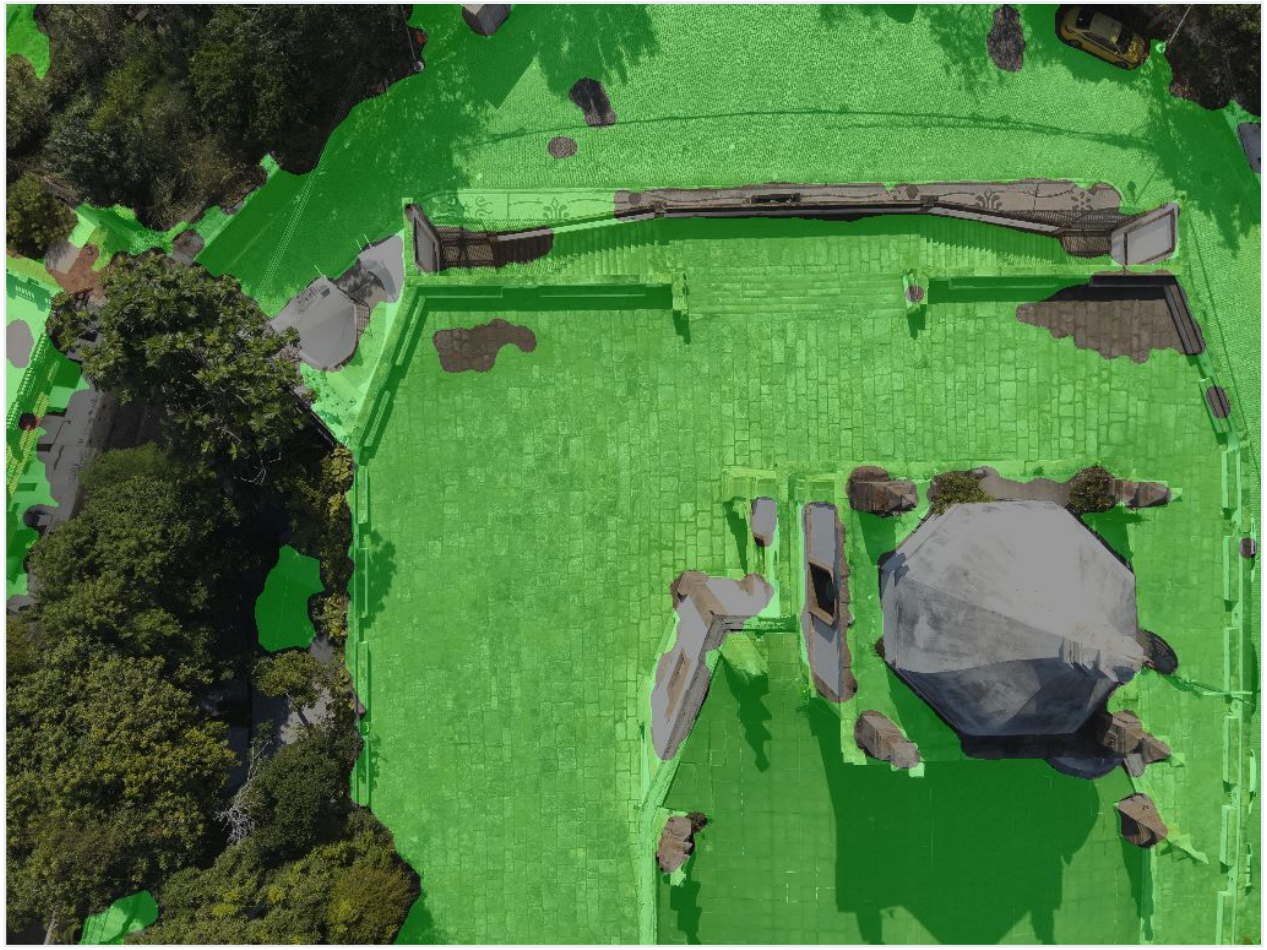
Aprimoramento 2: Segmentação Semântica

Identificação automática das áreas transitáveis no pipeline

- **Antes**: dependia de mapas segmentados externamente
- **Agora**: modelos U-Net pré-treinados (ResNet-50/101, EfficientNet-B4/B5) executados dentro do módulo de pré-processamento
- **Treinados no Semantic Drone Dataset**: 23 classes por pixel, agrupadas em transitável e não transitável
- **Modelo como hiperparâmetro**: troca-se o backbone sem alterar os demais módulos, equilibrando acurácia (mIoU) e tempo de inferência

Identificação de áreas transitáveis

Máscara sobre imagem real de drone (EfficientNet-B4)



Em verde: região classificada como transitável

Modelo	mIoU	Tempo (s)
ResNet-50	0,3933	0,72
ResNet-101	0,4032	0,52
EfficientNet-B4	0,5012	0,71
EfficientNet-B5	0,4819	0,57

EfficientNet-B4: melhor mIoU (0,5012), com inferência abaixo de 1 s.

Aprimoramento 3: Integração ROS 2 + Gazebo

Fechando o ciclo entre planejamento e execução

- **Publicação (ros_bridge)**: imagem anotada (sensor_msgs/Image) e trajetória (nav_msgs/Path); waypoints convertidos de pixels para metros
- **Consumo (ugv_bridge)**: o path_follower_node dirige o robô por comandos de velocidade (geometry_msgs/Twist), guiado pela odometria
- **Plataforma**: UGV Clearpath Jackal simulado no Gazebo, com odometria fundida por um filtro de Kalman estendido (EKF)
- **Status atual**: a integração está montada e o simulador executa; por ora, o robô segue apenas trajetórias simples de teste ('toy'), e a execução completa da rota planejada ainda está em ajuste

Aprimoramento 3: Integração ROS 2 + Gazebo

Etapas no planejamento da trajetória e exemplo de waypoint

Etapa 1:

Loading image: data/rgb/test_track.png

Image size: 400×400

Preprocessing with 'hsv_threshold' …

Walkable pixels: 62.3%

Nodes: 62,300 | Edges: 246,812

Start: pixel (60,60) → node 24060

End: pixel (340,340) → node 136340

Etapa 2:

Running a_star …

Path nodes: 281

Total cost: 396.98

Explored/iter: 18452

Time: 0.4123s

Robot waypoints: 281

Real distance: 19.85

Annotated image saved to: output/a_star_result.png

Etapa 3:

Publishing to ROS ('ros2') …

Image published to: /path_planning/annotated_image

Path published to: /path_planning/path

Exemplo de waypoint:

```
poses:
- header:
    frame_id: map
  pose:
    position:
      x: 3.2
      y: -1.05
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.3826834
      w: 0.9238795
```

Simulação do movimento no Gazebo

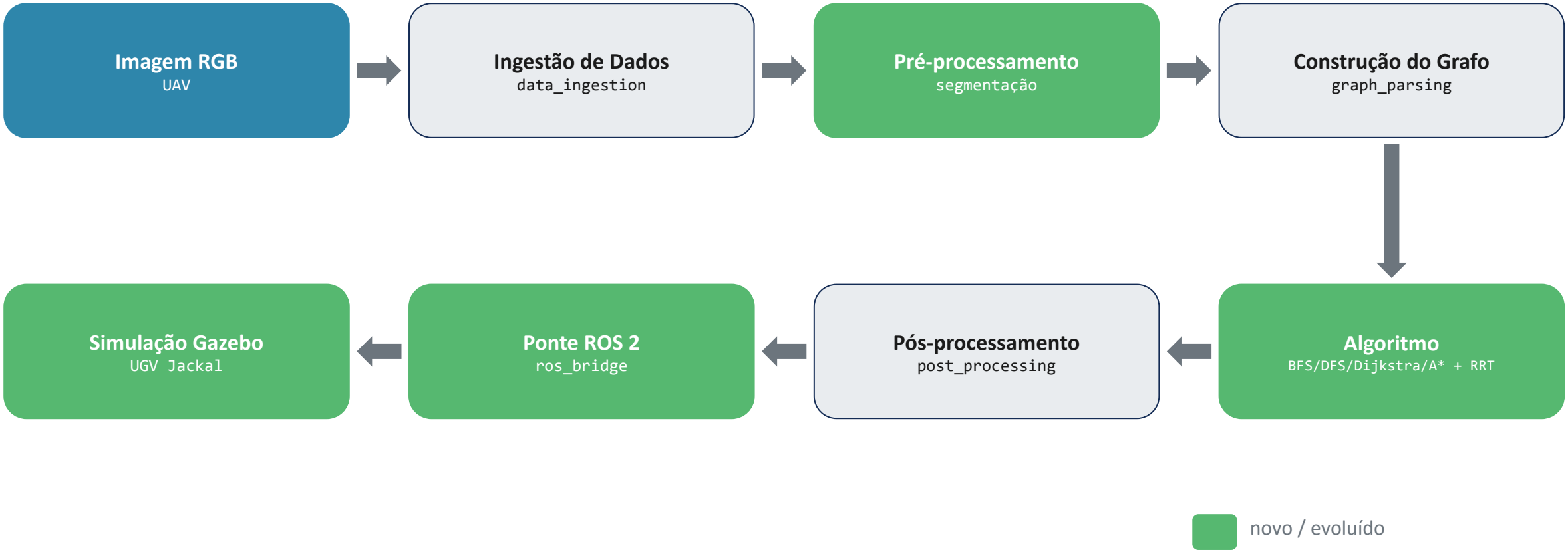
Simulador e integração em funcionamento; por ora, com trajetórias simples de teste



UGV Clearpath Jackal no Gazebo; à direita, o painel de teleoperação publicando em cmd_vel, o mesmo tópico consumido pela integração.

Pipeline do sistema (atualizado)

Em verde, o que evoluiu desde a VE



Próximos passos

O que segue para a versão final

- **Execução completa no simulador**: fazer o UGV percorrer integralmente a trajetória planejada, hoje validada apenas com trajetórias simples de teste ('toy')
- **Avaliação quantitativa no Gazebo**: medir a aderência da trajetória executada ao caminho planejado e o efeito da resolução do grafo e do fator metros-por-pixel
- **Ambientes dinâmicos**: estender o sistema para obstáculos que surgem, somem ou se movem, exigindo replanejamento em tempo real

Em resumo: da especificação ao UGV em simulação

- **RRT**: planejamento por amostragem somado ao catálogo de algoritmos
- **Segmentação semântica**: modelos U-Net identificam as áreas transitáveis no pipeline
- **ROS 2 + Gazebo**: integração montada e simulador em funcionamento; execução da rota planejada validada, por ora, com trajetórias simples de teste

Obrigado.